AetherHound Project Guide and Final Verification

Software only covert channel detector with virtual and real room demonstrations

This guide explains how to present, operate, test, and extend AetherHound. The default demonstration is fully virtual and needs no Raspberry Pi, SDR, or microphone. An optional separate room profile supports a laptop microphone without changing the virtual model or its calibration.

Final Status

Area	Final status	Meaning
Virtual dashboard	Ready	Rich terminal interface is configured at 30 refreshes per second and shows acoustic state, scores, tonal evidence, and waterfalls.
Virtual validation	Passed	All 1,404 saved baseline patches stayed below calibrated guards. All 936 saved attack patches were detected by the combined detector.
Automated checks	Passed	23 project tests pass, including virtual source, detector separation, calibration profile, RF, sweep, and bearing maths checks.
Real room mode	Ready to calibrate	A separate room profile learns normal microphone conditions. It must be calibrated on the actual room before presenting it.
Hardware deployment	Integration path available	Audio and RF source abstractions exist, but real Raspberry Pi and SDR operation have not been validated in this software-only project.

Project Purpose

AetherHound is a defensive monitoring prototype for a room containing an air-gapped system. It looks for unusual structured acoustic or RF activity that may resemble a covert channel. The supplied demonstration represents normal room audio and a simulated pulsed carrier. The objective is to show that normal baseline conditions remain clean while the structured attack is escalated.

The project brief asks for DSP, a PyTorch autoencoder trained on normal emissions, and a fast hacker-style command line interface that displays spectrum activity. AetherHound implements those software requirements with Python, SciPy, PyTorch/ONNX Runtime, SQLite, and Rich. Rich is permitted by the brief as an alternative to curses.

Requirements Check

Brief requirement	Current implementation	Assessment
DSP and spectrograms	Bandpass and mains notch filtering feed a 512 point STFT. Audio becomes 64 by 257 spectrogram patches.	Met for virtual and microphone software paths.
Autoencoder trained on normal data	PyTorch autoencoder is exported to ONNX and loaded for live scoring.	Met. Calibration and inference now use the same fixed reference scale.
CLI or curses spectrum display	Rich live terminal UI has acoustic and optional RF waterfall panels, status, scores, and frequency evidence.	Met. It is a CLI dashboard, not a graphical desktop app.
30 plus FPS UI	UI refresh configuration is 30 Hz.	Configured and exercised in the demo. Formal performance testing on a Raspberry Pi is still required.
Alert and local buzzer	Red threat state is implemented. The software buzzer is optional with --buzzer.	Met for the software alert. Hardware buzzer wiring is not part of the current build.
Timestamp and frequency logs	SQLite security events log state transitions; tonal channel starts include frequency_hz.	Met for acoustic events.
RF hardware and rack localisation	Simulated RF works. RTL SDR and HackRF backends are adapters only.	Not proven on hardware. Single sensor output is not rack-level location.

Dashboard Guide

The dashboard is accurate for its stated purpose: it presents the internal detector state and signal measurements. It must not be interpreted as proof that every yellow pixel is a threat or that it can locate a physical rack without additional hardware.

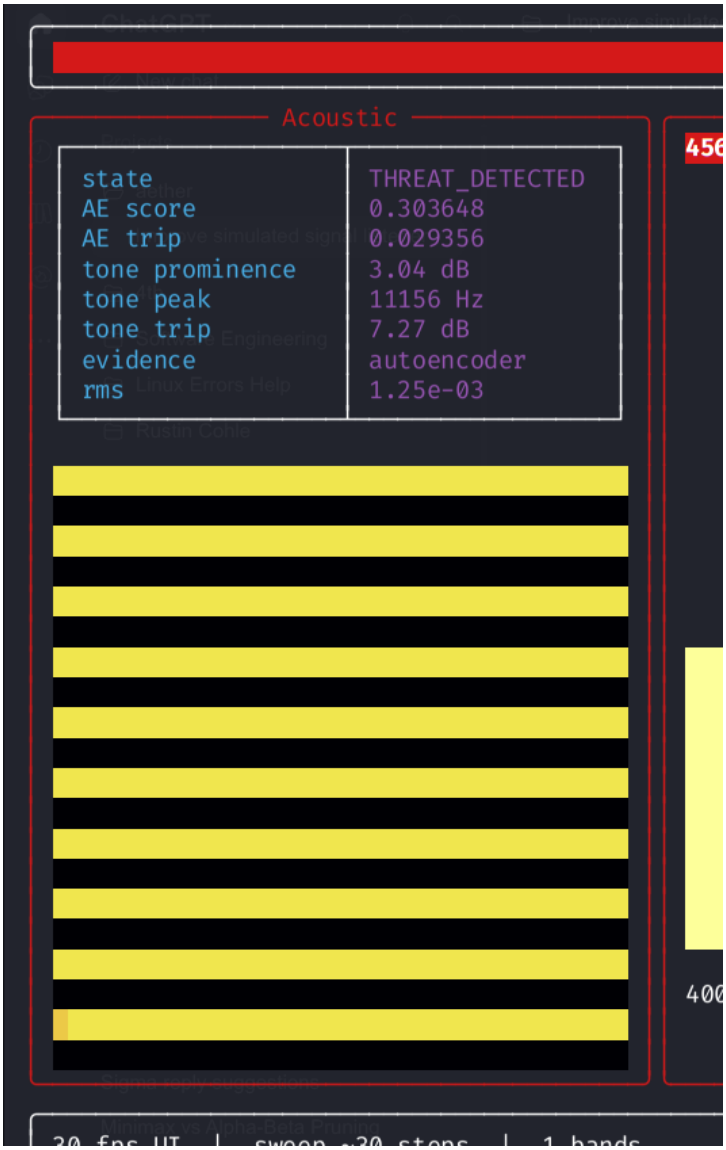


Figure 1 Acoustic dashboard during a threat state

What the Screenshot Shows

- The red header and THREAT DETECTED state mean the decision logic has escalated the current patch sequence.
- AE score is the autoencoder reconstruction error. AE trip is the calibrated error level that begins anomaly handling.
- Tone prominence measures how much a narrow frequency line rises above its nearby spectrum. Tone trip is the room or virtual threshold in decibels.
- Tone peak is the strongest narrow-band frequency at that instant. In the screenshot, the prominence is below the tone trip, so the autoencoder is the evidence source.
- RMS is the signal level in the current spectrogram patch. It helps distinguish silence from active input.

Yellow Bars

The yellow horizontal bars are the acoustic waterfall. Each row is one recent spectrogram patch, and each horizontal position represents a frequency region from approximately 200 Hz to 8 kHz. Yellow means high signal energy after the fixed display scale; black means low energy. Rows scroll as new patches arrive, so older history

moves upward. They are an intensity visualisation, not an alert by themselves. A loud song or broad noise can look yellow without being a confirmed covert channel.

Mic Off or Silent Room

Condition	Dashboard state	Expected display
Microphone sends zeros or room is below silence floor	NO SIGNAL	Grey header, score 0, tone 0, and evidence no signal.
No signal continues for about 5 seconds	SENSOR LOSS	Orange header and red acoustic border. This means the source is silent or unavailable, not that an attacker was found.
Input stream cannot open because a microphone is disconnected	Startup error or fallback	The program tries the system default device. Reconnect or select a valid input device before relying on monitoring.

Virtual Demonstration Runbook

Run these commands from the AetherHound project folder. The virtual paths are the primary project demonstration because they use only saved WAV data and generated RF signals.

Prepare and Verify

```
source .venv/bin/activate
python deploy/run_app.py --self-test
pytest -q
```

Expected result: three PASS lines from the self-test and 23 passed from pytest.

Clean Virtual Room

```
python deploy/run_app.py --scenario baseline
```

Expected result: a green CLEAN dashboard. The normal_room WAV loops as a virtual microphone. There should be no red threat state.

Virtual Acoustic Attack

```
python deploy/run_app.py --scenario attack
```

Expected result: the state progresses through ARMED to THREAT DETECTED. The tonal detector later reports a channel near 4031 Hz. Press Ctrl C to stop the dashboard.

Virtual RF and Acoustic Attack

```
python deploy/run_app.py --scenario attack --simulated-rf --inject
```

Expected result: the acoustic attack is visible and the RF waterfall receives an injected simulated carrier. This does not access an SDR.

Mixed Scenario

```
python deploy/run_app.py --scenario mixed --duration 40
```

Expected result: the dashboard starts clean, then reaches the attack fixture after the baseline fixture has played. A 40 second duration is suitable for a repeatable demonstration.

Specific Supplied WAV Files

```
python deploy/run_app.py --audio-file "$(pwd)/data/baseline/live_room.wav" --duration 10
python deploy/run_app.py --audio-file "$(pwd)/data/baseline/normal_room.wav" --duration 10
python deploy/run_app.py --audio-file "$(pwd)/data/test/infected_attack.wav" --duration 10
```

The first two are expected to remain clean. The final WAV is expected to reach a threat state. Custom WAV input must be mono or stereo 48 kHz audio.

Real Room Profile Runbook

Real microphone mode is optional and intentionally separate from the virtual demonstration. The base ONNX model is not retrained when a room profile is created. The profile only saves thresholds for the actual microphone environment.

Choose a Microphone if Needed

```
python deploy/run_app.py --list-devices
```

If the default microphone is not correct, note the input-device number and include it during room calibration.

Calibrate Normal Conditions

```
python train/calibrate_room.py --seconds 45
```

During recording, keep the sounds you want to regard as normal present: ordinary room/fan noise, speech, and a short sample of normal music. The command writes models/profiles/room.json. It does not overwrite the virtual threshold files.

Start the Real Room Dashboard

```
python deploy/run_app.py --source microphone --profile room
```

Expected result: normal conditions that were included in calibration should remain CLEAN. Absolute silence becomes NO SIGNAL and later SENSOR LOSS. If a normal repeated sound produces an alert, recalibrate with that sound present; do not alter the virtual profile.

Optional Named Device

```
python train/calibrate_room.py --seconds 45 --device 3
python deploy/run_app.py --source microphone --profile room
```

Replace 3 with the input-device number shown by --list-devices. The dashboard itself uses the configured/default microphone after calibration.

Important Real Room Limit

The saved validation result is a synthetic-fixture result, not a guarantee for every real song, voice, fan, or room. A room profile reduces false positives for conditions it has observed. It should be recalibrated after a microphone, room layout, or normal noise pattern changes.

ML Model Status and Accuracy

Item	Current value	Interpretation
Deployment model	models/autoencoder.onnx, about 65 MB	ONNX Runtime model used by the detector.
Training checkpoint	models/autoencoder.pt, about 65 MB	PyTorch training artifact.
Input	64 by 257 magnitude spectrogram	A short time-frequency patch from the audio DSP path.
Architecture	16,448 to 512 to 64 bottleneck to 512 to 16,448	Fully connected autoencoder with ReLU layers and sigmoid decoder.
Training	Adam, learning rate 0.001, MSE, 15 epochs	Trained only on baseline patches using a fixed reference scale.
Virtual AE trip	0.029356	Operational reconstruction-error threshold after applying the configured margin.
Virtual tone trip	7.275 dB	Secondary narrow-band guard calibrated from baseline patches.
Saved fixture result	0 of 1,404 baseline breaches; 936 of 936 attack patches detected	Strong in-project validation, but not a general real-world accuracy percentage.

The detector is hybrid. The autoencoder finds patterns that reconstruct poorly compared with normal data. A separate tonal-prominence measurement looks for narrow spectral lines commonly associated with pulsed acoustic channels. A threat can be raised by either guard, and a tonal-channel event includes its frequency in the SQLite log.

The 100 percent attack-fixture detection figure should be presented carefully: it applies to the supplied 936 synthetic attack patches and the saved baseline set used for calibration. It does not prove 100 percent accuracy against unknown real attacks or all possible music and environmental noise. Real-world evaluation needs separately labelled recordings, multiple rooms, multiple microphones, false-positive measurement, and hardware performance tests.

How the Project Was Built

- Synthetic normal and infected WAV fixtures were generated for a safe reproducible demonstration.
- The DSP path applies bandpass and mains-harmonic filtering, then uses a short-time Fourier transform to create spectrogram patches.
- The autoencoder learns to reconstruct normal patches. Large reconstruction error is anomaly evidence.
- The ONNX export keeps inference lightweight and avoids requiring PyTorch during normal monitoring.
- The detector uses hysteresis states CLEAN, ARMED, THREAT DETECTED, NO SIGNAL, and SENSOR LOSS to avoid treating a single frame as a final decision.
- The Rich terminal dashboard renders acoustic and optional RF waterfalls, while SQLite retains event state, timestamps, scores, descriptions, and tonal frequency where available.
- Virtual and real-room calibration profiles are kept separate so a physical microphone experiment never changes the verified virtual demonstration.

Optional Raspberry Pi and Hardware Integration

The current requested deliverable is software-first and does not require hardware. The repository nevertheless contains an integration path for a later edge-device build. Treat the following as an engineering plan, not as completed or validated deployment work.

Existing Integration Points

Hardware role	Existing software point	What remains
USB microphone	LiveAudioSource through sounddevice and --source microphone	Select stable device, calibrate room profile, measure CPU load and latency.
RTL SDR	RTLSDRBackend in capture/rf_source.py	Install radio driver and pyrtlsdr, set RF configuration, create an RF baseline.
HackRF	HackRFBackend in capture/rf_source.py	Install driver/binding, set gain and sample rate, create an RF baseline.
Edge computer or Raspberry Pi	Python runner and ONNX Runtime inference	Benchmark 30 Hz UI and RF sweep throughput on the chosen board.
Multiple antennas or sensors	Bearing and sweep modules	Design sensor geometry, synchronisation, and a validated localisation method.

Suggested Hardware Bring Up

- Use a supported Linux edge device with a 48 kHz USB microphone and either an RTL SDR or HackRF. Connect only hardware authorised for the deployment environment.
- Install the base project dependencies and, only for real RF use, the optional RF dependencies and vendor drivers.

```
pip install -r requirements.txt
pip install -r requirements-rf.txt
```

- Set audio.device to the correct microphone input number. Set rf.enabled to true and rf.backend to rtlsdr or hackrf in config.toml. Keep the virtual configuration as a separate copy for presentations.
- Create a room profile with calibrate_room.py. Then create an RF baseline in an authorised quiet environment.

```
python train/calibrate_room.py --seconds 45
python capture/record_rf.py --sweeps 100
python deploy/run_app.py --source microphone --profile room
```

- Run controlled tests with benign simulated carriers only. Record false positives, missed detections, temperature, CPU use, UI refresh rate, and RF sweep time before making any security claim.

Hardware Limits

A single microphone or single RF receiver can report anomaly evidence and a frequency, but it cannot reliably identify an infected server rack. Rack-level localisation needs multiple calibrated sensors or directional/steered antennas, controlled geometry, synchronisation, and testing against multipath reflections. This capability is outside the validated scope of the current project.

Demo Day Script

Use this short sequence when presenting the project. It separates proven virtual behaviour from the optional calibrated real-room experiment.

Step	Command or action	What to explain
1	python deploy/run_app.py --scenario baseline	This is the normal virtual room. The dashboard remains CLEAN.
2	python deploy/run_app.py --scenario attack --simulated-rf --inject	The structured virtual signal drives ARMED then THREAT DETECTED; the dashboard exposes AE and tonal evidence.
3	Show Figure 1 metrics	Yellow waterfall energy is visual context. The red state comes from detection logic, not colour alone.
4	Optional: python deploy/run_app.py --source microphone --profile room	This is separately calibrated for the current room. Normal calibrated conditions should remain CLEAN.
5	Show pytest -q and --self-test output	The software has automated checks and fixture validation; real-world accuracy still needs field evaluation.

Quick Troubleshooting

Symptom	Cause	Action
File not found for a custom WAV	Example path was typed literally or file does not exist	Use one of the supplied data paths or give the full path to an existing 48 kHz WAV.
Normal music shows threat in microphone mode	Virtual calibration was used or music was not represented during room calibration	Create or refresh the room profile while the normal music is present.
Mic off shows NO SIGNAL or SENSOR LOSS	The silence gate sees no usable input	This is correct behaviour. Check the microphone only if monitoring should be active.
Room profile cannot be found	Calibration has not yet created models/profiles/room.json	Run python train/calibrate_room.py --seconds 45.
RF hardware does not start	Driver, permission, device, or optional package is missing	Return to simulated RF for the demo; troubleshoot the hardware separately before claiming deployment.